Neural Approximation of Linear Programming Solutions

Abstract

This project explores the use of neural networks to approximate solutions to LPPs. A dataset of randomly generated LP instances is created, and a neural network is trained to predict optimal solutions obtained via the simplex algorithm. The model is evaluated in terms of feasibility, objective accuracy, and computational efficiency. Results show that while the neural network provides near-constant inference time, it trades off accuracy compared to exact solvers.

1 Introduction

Linear programming is a fundamental optimization technique used in various domains such as operations research, economics, and engineering. Classical methods like the simplex algorithm compute exact solutions but can become computationally expensive for large-scale problems.

This project investigates whether a neural network can learn to approximate LP solutions, providing faster inference at the cost of exact optimality.

2 Problem Formulation

We consider linear programs of the form:

$$\max \; c^T x \tag{1}$$

$$\text{subject to } Ax \leq b, \quad x \geq 0 \tag{2}$$

where:

- $A \in \mathbb{R}^{m \times n}$
- $b \in \mathbb{R}^m$
- $c \in \mathbb{R}^n$

The optimal solution x^* is computed using a simplex implementation.

3 Dataset Generation

Random LP instances are generated as follows:

- $A \sim \mathcal{U}(-5, 9)$
- $b \sim \mathcal{U}(0, 30)$
- $c \sim \mathcal{U}(-3, 7)$

Problems with invalid or degenerate solutions are filtered out. All inputs are padded to a fixed size of 7×7 to allow batching.

4 Model Architecture

The model follows a DeepSets-style architecture to handle sets of constraints.

Each constraint row (a_i, b_i) is processed independently using a shared function ϕ:

- Linear layer
- Layer Normalization
- ReLU activation
- Linear layer
- Layer Normalization
- ReLU activation

The outputs are aggregated using:

- Mean pooling
- Max pooling

These features are concatenated with:

- Objective vector c
- Optimization flag (max/min)

The final prediction is produced using a multilayer perceptron (MLP).

5 Training and Loss Function

The model is trained using a combination of regression and constraint-based losses:

$$\mathcal{L} = \|x_{\text{pred}} - x^*\|^2 + \lambda_1 \cdot \max(0, Ax - b)^2 + \lambda_2 \cdot \max(0, -x)^2 \tag{3}$$

This encourages:

- Accurate prediction of optimal solutions
- Feasibility with respect to constraints
- Non-negativity of variables

6 Evaluation Metrics

Constraint Violation:

$$\max(0, Ax - b) + \max(0, -x) \tag{4}$$

Objective Gap:

$$\frac{|c^T x_{\text{pred}} - c^T x^*|}{|c^T x^*| + 1} \tag{5}$$

7 Results

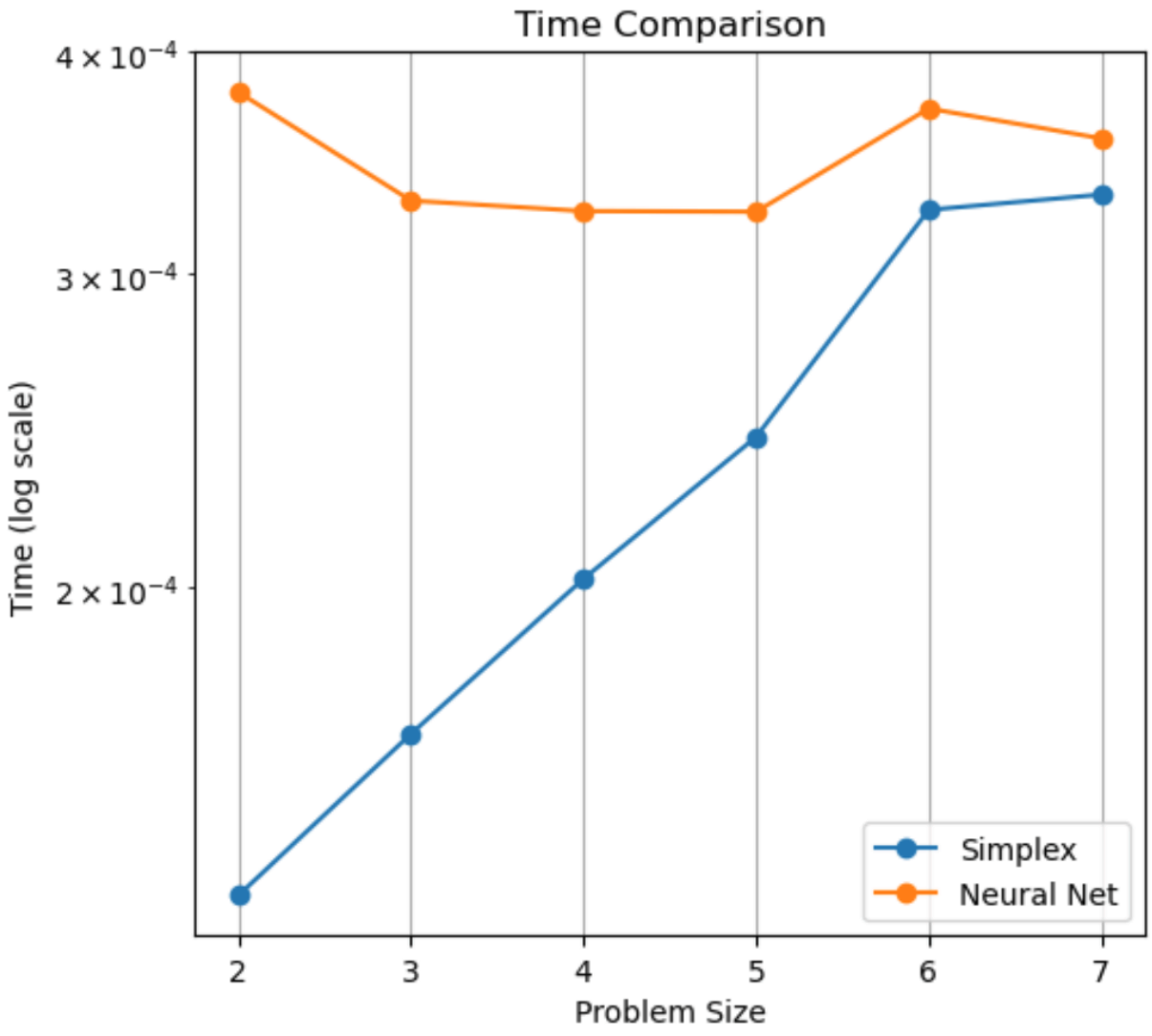

Figure 1: Time Plot

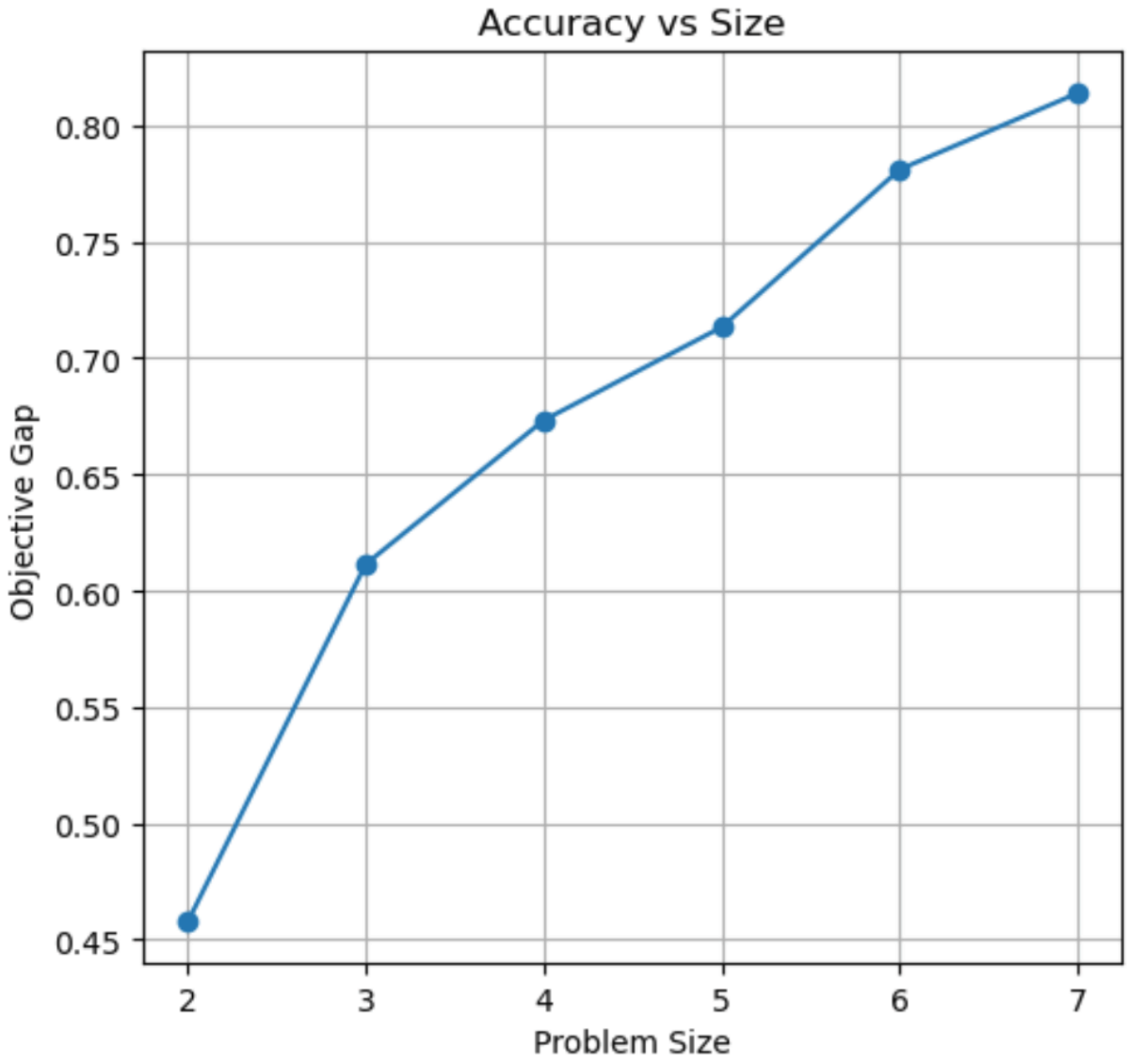

Figure 2: Accuracy Plot

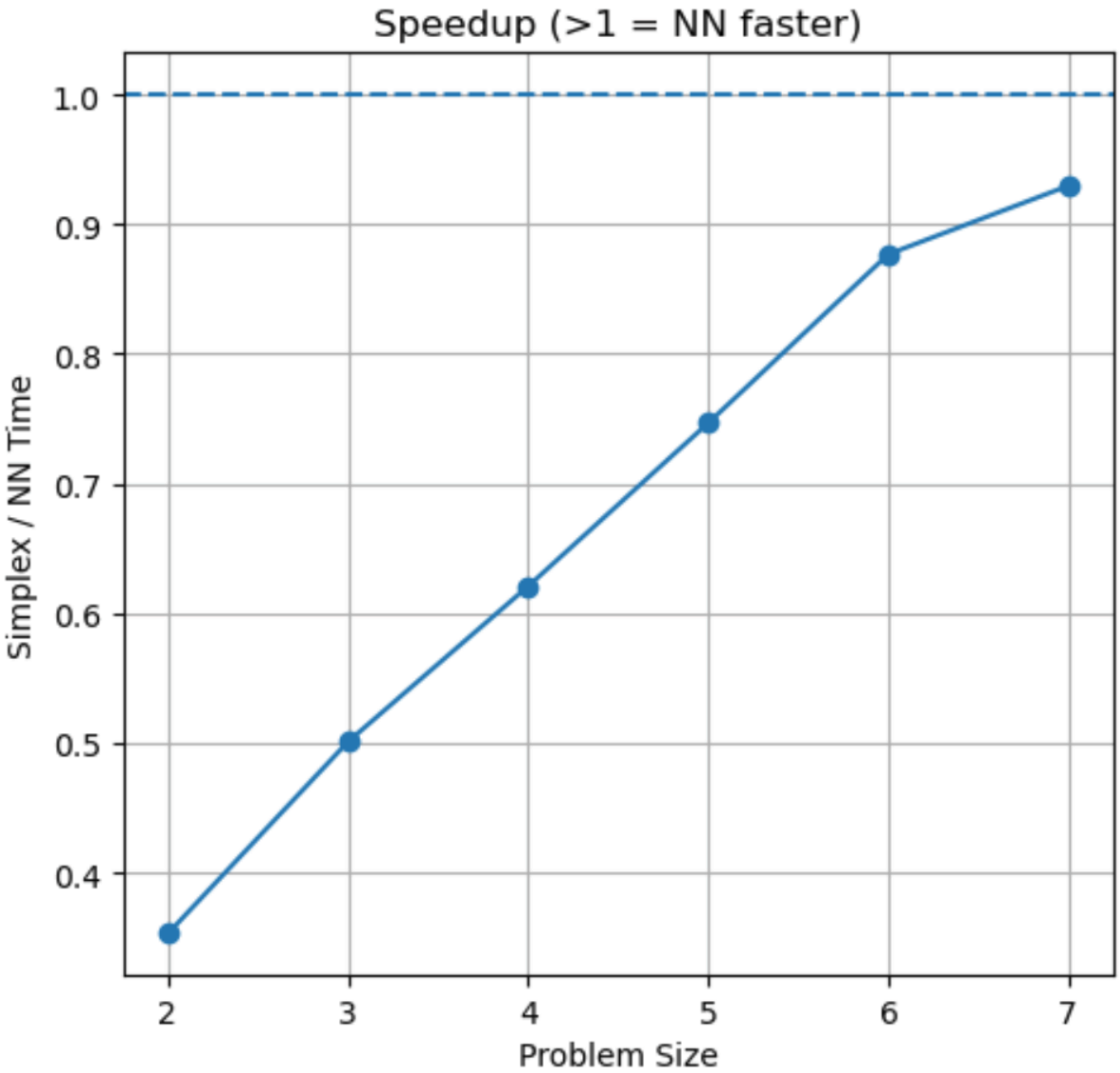

Figure 3: Speedup Plot

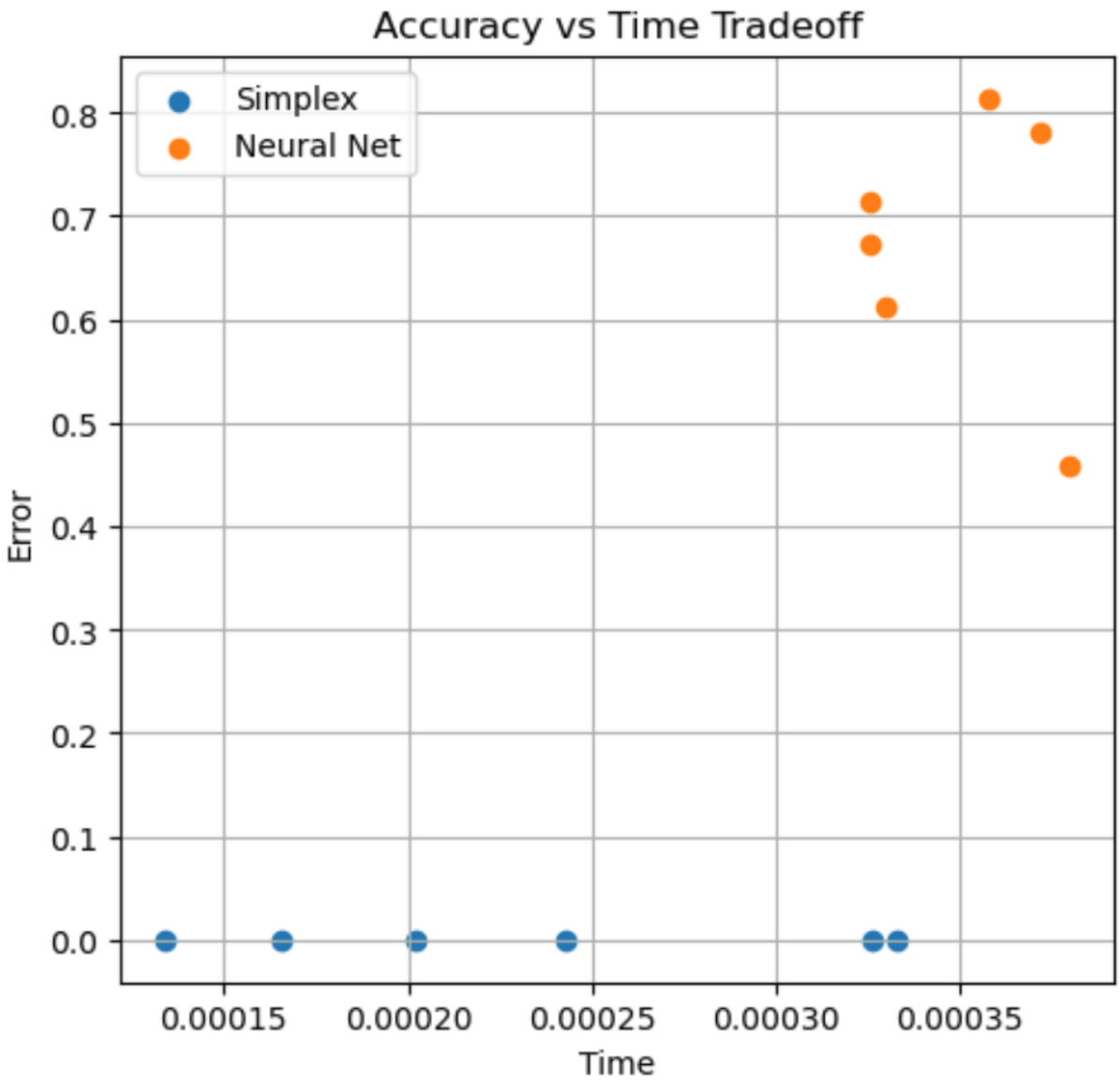

Figure 4: Time Accuracy Trade off

Observations:

- Neural network inference time remains nearly constant
- Simplex time increases with problem size
- Neural network solutions are approximately feasible
- Objective accuracy decreases as problem size increases

8 Conclusion

Neural networks can approximate solutions to linear programs with reasonable accuracy and significantly faster inference. While they do not replace exact solvers, they offer a useful tradeoff between speed and optimality.